

# Das Python-Tutorial

**Release:** 3.3

**Date:** Oktober 08, 2017

Python ist eine einfach zu lernende, aber mächtige Programmiersprache mit effizienten abstrakten Datenstrukturen und einem einfachen, aber effektiven Ansatz zur objektorientierten Programmierung. Durch die elegante Syntax und die dynamische Typisierung ist Python als interpretierte Sprache sowohl für Skripte als auch für schnelle Anwendungsentwicklung (Rapid Application Development) hervorragend geeignet.

Der Python-Interpreter und die umfangreiche Standardbibliothek sind als Quelltext und in binärer Form für alle wichtigen Plattformen auf der Webseite <http://www.python.org> frei verfügbar, und können frei weiterverbreitet werden. Auf der gleichen Seite finden sich Distributionen von Drittanbietern, Verweise auf weitere freie Module, Programme und Werkzeuge, sowie zusätzliche Dokumentation.

Der Python-Interpreter kann auf einfache Weise um neue Funktionen und Datentypen erweitert werden, die in C oder C++ (oder andere Sprachen, die sich von C aus ausführen lassen) implementiert sind. Auch als Erweiterungssprache für anpassbare Applikationen ist Python hervorragend geeignet.

Dieses Tutorial stellt die Grundkonzepte und Eigenschaften der Sprache und des Systems Python vor. Zwar ist es hilfreich, einen Python-Interpreter griffbereit zu haben, um praktische Erfahrungen zu sammeln, aber alle Beispiele sind eigenständig, so dass das Tutorial auch offline gelesen werden kann.

Eine Beschreibung der Standardobjekte und -module ist in der Referenz der [Pythonbibliothek \(Python Library Reference\)](#) zu finden. Das [Python Referenzhandbuch \(Python Reference Manual\)](#) bietet eine formellere Definition der Sprache. Um Erweiterungen in C oder C++ zu schreiben, sollte man sich [Extending and Embedding the Python Interpreter](#) sowie die [Python/C API Reference](#) anschauen. Es gibt auch zahlreiche Bücher, die Python tiefergehend behandeln.

Ziel dieses Tutorials ist nicht die umfangreiche und vollständige Behandlung sämtlicher Sprachmerkmale von Python, sondern die Vermittlung der wichtigsten Eigenschaften, um einen Eindruck von dem zu bekommen, was Python ist. Nach der Lektüre

Lage sein, Python-Module und -Programme zu schreiben. Außerdem verfügt man dann über die notwendigen Grundlagen, um sich mit weiteren Modulen der Standardbibliothek zu beschäftigen, die in der [Python Library Reference](#) beschrieben werden.

Es lohnt sich ebenfalls das [Glossar](#) durchzugehen.

- 1. Um dich auf den Geschmack zu bringen
- 2. Verwendung des Python-Interpreters
  - 2.1. Aufrufen des Interpreters
    - 2.1.1. Übergabe von Argumenten
    - 2.1.2. Interaktiver Modus
  - 2.2. Der Interpreter und seine Umgebung
    - 2.2.1. Fehlerbehandlung
    - 2.2.2. Ausführbare Python-Skripte
    - 2.2.3. Kodierung von Quellcode
    - 2.2.4. Die interaktive Startup-Datei
    - 2.2.5. Die Customization Module
- 3. Eine informelle Einführung in Python
  - 3.1. Benutzung von Python als Taschenrechner
    - 3.1.1. Zahlen
    - 3.1.2. Zeichenketten (Strings)
    - 3.1.3. Über Unicode
    - 3.1.4. Listen
  - 3.2. Erste Schritte zur Programmierung
- 4. Mehr Werkzeuge zur Ablaufsteuerung
  - 4.1. `if`-Anweisungen
  - 4.2. `for`-Anweisungen
  - 4.3. Die Funktion `range()`
  - 4.4. `break`- und `continue`-Anweisungen und der `else`-Zweig bei Schleifen
  - 4.5. `pass`-Anweisungen
  - 4.6. Funktionen definieren
  - 4.7. Mehr zum Definieren von Funktion
    - 4.7.1. Standardwerte für Argumente
    - 4.7.2. Schlüsselwortargumente
    - 4.7.3. Beliebige lange Argumentlisten
    - 4.7.4. Argumentlisten auspacken
    - 4.7.5. Lambda-Form - anonyme Funktion
    - 4.7.6. Dokumentationsstrings
  - 4.8. Intermezzo: Schreibstil
- 5. Datenstrukturen
  - 5.1. Mehr zu Listen
    - 5.1.1. Benutzung von Listen als Stack
    - 5.1.2. Benutzung von Listen als Queue
    - 5.1.3. List Comprehensions
    - 5.1.4. Verschachtelte List Comprehensions

- 5.2. Die `del`-Anweisung
- 5.3. Tupel und Sequenzen
- 5.4. Mengen
- 5.5. Dictionaries
- 5.6. Schleifentechniken
- 5.7. Mehr zu Bedingungen
- 5.8. Vergleich von Sequenzen mit anderen Typen
- 6. Module
  - 6.1. Mehr zum Thema Module
    - 6.1.1. Module als Skript aufrufen
    - 6.1.2. Der Modul-Suchpfad
    - 6.1.3. "Kompilierte" Python-Dateien
  - 6.2. Standardmodule
  - 6.3. Die `dir()`-Funktion
  - 6.4. Pakete
    - 6.4.1. \* aus einem Paket importieren
    - 6.4.2. Referenzen innerhalb des Paketes
    - 6.4.3. Pakete in mehreren Verzeichnissen
- 7. Eingabe und Ausgabe
  - 7.1. Ausgefallene Ausgabeformatierung
    - 7.1.1. Alte Zeichenkettenformatierung
  - 7.2. Lesen und Schreiben von Dateien
    - 7.2.1. Methoden von Dateiobjekten
    - 7.2.2. Das `pickle`-Modul
- 8. Fehler und Ausnahmen
  - 8.1. Syntaxfehler
  - 8.2. Ausnahmen
  - 8.3. Ausnahmen behandeln
  - 8.4. Ausnahmen auslösen
  - 8.5. Benutzerdefinierte Ausnahmen
  - 8.6. Aufräumaktionen festlegen
  - 8.7. Vordefinierte Aufräumaktionen
- 9. Klassen
  - 9.1. Ein Wort zu Namen und Objekten
  - 9.2. Gültigkeitsbereiche und Namensräume in Python
    - 9.2.1. Beispiel zu Gültigkeitsbereichen und Namensräumen
  - 9.3. Eine erste Betrachtung von Klassen
    - 9.3.1. Syntax der Klassendefinition
    - 9.3.2. Klassenobjekte
    - 9.3.3. Instanzobjekte
    - 9.3.4. Methodenobjekte
  - 9.4. Beiläufige Anmerkungen
  - 9.5. Vererbung
    - 9.5.1. Mehrfachvererbung
  - 9.6. Private Variablen



- 9.7. Kleinkram
- 9.8. Ausnahmen sind auch Klassen
- 9.9. Iteratoren
- 9.10. Generatoren
- 9.11. Generator Ausdrücke
- 10. Eine kurze Einführung in die Standardbibliothek
  - 10.1. Schnittstelle zum Betriebssystem
  - 10.2. Platzhalter in Dateinamen
  - 10.3. Argumente in der Befehlszeile
  - 10.4. Umleitung von Fehlermeldungen und Programmabbruch
  - 10.5. Muster in Zeichenketten
  - 10.6. Mathematik
  - 10.7. Zugriff auf das Internet
  - 10.8. Datum und Uhrzeit
  - 10.9. Datenkompression
  - 10.10. Performancemessung
  - 10.11. Qualitätskontrolle
  - 10.12. Batteries Included
- 11. Eine kurze Einführung in die Standardbibliothek - Teil II
  - 11.1. Ausgabeformatierung
  - 11.2. Templating
  - 11.3. Arbeit mit strukturierten binären Daten
  - 11.4. Multi-threading
  - 11.5. Logging
  - 11.6. Weak References
  - 11.7. Werkzeuge zum Arbeiten mit Listen
  - 11.8. Dezimale Fließkomma-Arithmetik
- 12. Wie geht's weiter?
- 13. Interaktive Eingabe-Bearbeitung und Ersetzung des Verlaufs
  - 13.1. Zeilenbearbeitung
  - 13.2. Ersetzung des Verlaufs
  - 13.3. Tastenkombinationen
  - 13.4. Alternativen zum Interaktiven Interpreter
- 14. Fließkomma-Arithmetik: Probleme und Einschränkungen
  - 14.1. Darstellungsfehler
- 15. Zur Übersetzung
- 16. Autoren der Übersetzung
- 17. Möglichkeiten der Mitarbeit
- 18. Glossar

From AI to APIs, we have everything you need to personalize each and every search. [Visit the Hub](#)